

Claude Code from Source: Key Patterns, Tricks, and Novel Ideas

A Technical Distillation of Anthropic’s AI Coding Agent Architecture

Scott Mutchler

2026-05-07

Table of contents

What This Document Is	1
The Surprising Starting Point	2
The Six-Layer Architecture	2
The Generator Loop: One While(True) to Rule Them All	4
State: The Two-Tier Architecture	5
Prompt Cache Economics: The Constraint That Shapes Everything	6
Speculative Tool Execution: 16% Free	7
The Tool System: 14 Steps and Self-Describing Objects	7
Concurrent Execution: Order Preservation Through Buffering	8
Fork Agents: Cache Sharing as Architecture	9
Context Management: Four Layers Before Giving Up	11
The Terminal UI Nobody Sees	11
Memory Without a Database	12
The Five Architectural Bets	13
What Makes This Architecture Worth Studying	14
Closing Thought	15

What This Document Is

Anthropic’s Claude Code CLI was reverse-engineered from source maps that leaked via npm. Those source maps exposed the actual TypeScript behind a production AI agent used by hundreds of thousands of developers. Someone built a 18-chapter technical book from that analysis. This document distills

the most surprising, instructive, and transferable ideas from that book into a single readable artifact.

The goal is not completeness. It is to answer: *what is genuinely novel here, and why did they build it this way?*

The Surprising Starting Point

Claude Code is not written in Python. It is not a thin wrapper around API calls. It is 135,000 lines of TypeScript running on Bun, structured in six architectural layers, with a React-based terminal UI, an async generator as its core agent loop, a prompt cache economics model that shapes nearly every design decision, and a memory system that deliberately avoids vector databases.

The starting premise of the architecture is that **every millisecond and every token costs money at scale**. This is not an abstraction. It is the constraint that explains why sticky boolean latches exist, why fork agents thread their parent's rendered system prompt instead of recomputing it, why the terminal renderer uses packed integer arrays instead of objects, and why the bootstrap pipeline fires I/O as module-level side effects during import evaluation.

When you understand that constraint, the architecture stops looking clever and starts looking *inevitable*.

The Six-Layer Architecture

The six layers are not a planned design. They emerged from shipping failures. Each layer exists because production revealed something that could not live anywhere else. The REPL handles display and input. The query loop handles model interaction and tool orchestration. The tool system handles execution. The state layer handles everything that needs to persist within or across turns. The memory system handles everything that needs to survive across sessions. The hooks system handles everything that third parties need to intercept.

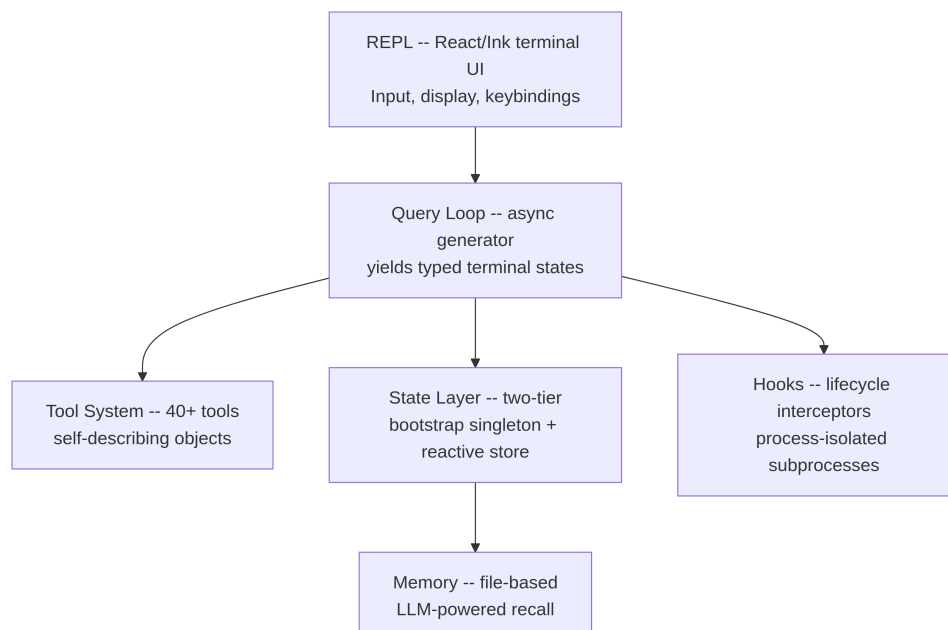


Figure 1: The six architectural layers of Claude Code, from user input to tool execution

i Note

This is not a microservices architecture. There is no network. All six layers run in a single Bun process, communicating through shared state and function calls. The layering is logical, not physical.

The Generator Loop: One While(True) to Rule Them All

The heart of Claude Code is a single async generator function called `query()`. It is 1,730 lines long. It contains a `while(true)` loop. Every interaction – REPL sessions, SDK calls, sub-agents, headless batch runs – flows through this one function.

This is a controversial design choice. A single 1,730-line function is an immediate code smell in conventional software engineering. But the alternative – breaking it into smaller functions – would scatter the state machine across files and make it nearly impossible to understand what actually happens during a turn.

The genius is in the return type. The generator does not return a boolean or throw exceptions. It *yields* typed terminal states:

- `blocking_limit` – token budget exhausted
- `model_error` – the API failed
- `aborted_streaming` – user hit Escape during streaming
- `prompt_too_long` – context exceeded limits
- `completed` – normal finish
- `stop_hook_prevented` – a hook blocked the response
- `hook_stopped` – a hook terminated the session
- `max_turns` – turn limit reached

Ten distinct terminal states, all encoded in the generator’s yield type. The caller does not need try/catch logic. It just iterates the generator and pattern-matches on what comes out. This is discriminated unions as architecture.

Similarly, continuation reasons are typed. When the loop needs to keep going, it yields a `continuation` with one of seven reasons: `reactive_compact_retry`, `max_output_tokens_escalate`, `token_budget_continuation`, and others. Each continuation reason is a named, documented decision, not a silent fall-through.

The pattern that emerges is: **encode decisions in the type system, not in exception handlers.**

State: The Two-Tier Architecture

Claude Code has two state systems that must never mix.

The first is a bootstrap singleton called **STATE**. It initializes before React mounts. It carries 80+ fields: the current working directory, session ID, parent session ID, cost metrics, telemetry handles, model configuration, and several cache optimization flags. This singleton is loaded in a strict dependency order: it cannot import React, cannot import the reactive store, and this constraint is enforced by ESLint rules on the import graph.

The second is a reactive store called **AppState**. It is 34 lines of code. No Redux, no Zustand, no middleware. Just a closure over a mutable variable, a **Set** of listener callbacks, and an **Object.is** equality check before notifying. The simplicity is intentional. The store does exactly one thing: notify subscribers when something changes.

What makes this architecture interesting is how the two systems bridge. A single **onChange** callback fires before listener notification, giving infrastructure code a chance to react to state changes without becoming a subscriber. And the **AppState** is typed as **DeepImmutable<>** – you cannot mutate it accidentally. Changes require calling **setAppState()**, which replaces the entire object.

The Sticky Latch Pattern

Here is one of the most instructive tricks in the codebase. Prompt caching requires that the system prompt be byte-identical across requests. But some system prompt sections are gated on feature flags, and feature flags can change during a session as remote configuration warms up from cold to hot.

The naive implementation lets feature flags toggle sections in and out, which changes the system prompt, which busts the cache, which costs tens of thousands of tokens on every affected request.

The fix is a **sticky latch**: a three-state boolean that starts as **null**, transitions to **true** when first activated, and never transitions back to **false**. Once a cache-relevant feature is active, it stays active for the entire session. The

flag latch sacrifices correctness-at-the-margins (a feature could stay activated past when it should) for cache stability. The economics make this obvious: the cost of a wrong feature flag staying on is near zero. The cost of a cache miss on a 50,000-token prefix is real money.

Prompt Cache Economics: The Constraint That Shapes Everything

To understand why Claude Code is built the way it is, you need to understand one number: **prompt caching gives a 90% token discount on prefixes that are byte-identical across requests.**

The system prompt for Claude Code is long. It includes CLAUDE.md project files, user memories, tool descriptions, beta headers, and instructions. At scale, this prefix can reach 50,000–80,000 tokens. Without caching, every turn reprocesses those tokens. With caching, turns after the first pay roughly 10% of that cost.

This single fact explains:

- Why feature flag sections use sticky latches (toggling breaks the prefix)
- Why fork agents receive the parent’s *rendered* system prompt instead of recomputing it (recomputation risks divergence)
- Why the date in the system prompt is memoized per session (midnight date change = cache bust for all active sessions)
- Why there is a `DANGEROUS_uncachedSystemPromptSection()` function with a deliberately alarming name (loud naming forces documentation of why this section must bust cache)
- Why the system prompt has an explicit `=== DYNAMIC BOUNDARY ===` marker separating globally-cacheable sections from per-session sections

The dynamic boundary is particularly elegant. Everything before the marker is shared across all users (global cache scope). Everything after is per-session. This prevents the combinatorial explosion of 2^N cache variants from runtime conditionals scattered through the prompt.

! Important

One space character in the wrong place invalidates the cache for the entire user fleet on that prefix. The architecture treats byte-identity as

a first-class constraint, not an implementation detail.

Speculative Tool Execution: 16% Free

Here is a trick that saves real time without any user-visible change.

When the model returns a response, it often includes multiple tool calls. Conventionally, the system waits for the full response before executing anything. But Claude Code starts executing *read-only* tool calls the moment their `tool_use` blocks finish parsing in the stream – before the model has finished its complete response.

By the time the model finishes streaming, the read-only tools (file reads, grep operations, searches) have often already completed. The results are buffered and replayed in the order the tools were requested, not the order they completed, so the model’s reasoning remains coherent.

This is speculative execution applied to tool calls. The safety property that makes it work: read-only tools have no side effects, so speculating on them and discarding the results (if the response ends up not needing them) is harmless. Write tools are never speculatively executed.

Benchmark: 16% reduction in wall-clock time on tool-heavy sessions.

The Tool System: 14 Steps and Self-Describing Objects

Tools in Claude Code are not functions. They are objects that carry:

- A name and description (for the model)
- A Zod schema for input validation
- An `isConcurrencySafe()` method that takes *parsed input* (not just the tool type)
- A `checkPermissions()` method
- A `renderResult()` method for display
- Metadata flags: `isReadOnly`, `isDeferred`, `requiresNetwork`

The self-describing design means tools can be registered by MCP servers (external processes implementing a JSON-RPC protocol) and become indistinguishable from built-in tools after wrapping. There is no adapter layer.

The same 14-step execution pipeline handles both.

Those 14 steps are: tool lookup (with alias fallback) – abort check – Zod validation – semantic validation – speculative classifier start – input backfill – PreToolUse hooks – permission resolution – tool execution – result budgeting – PostToolUse hooks – new message construction – error handling.

The permission resolution alone has seven tiers: hook decision (final, cannot be overridden) – rule matching (glob patterns like `Bash(git *)` or `Edit(/src/**)`) – tool-specific check – mode-based default – interactive prompt – auto-mode classifier.

Concurrency Classification Per Input

The `isConcurrencySafe()` method takes the *parsed input*, not just the tool name. This is non-obvious but important. `Bash("ls -la")` is safe to run concurrently. `Bash("rm -rf /")` is not. The safety classification is input-dependent, not tool-dependent.

For bash commands specifically, the tool decomposes `cmd1 && cmd2 && cmd3` into sub-commands, classifies each against known-safe sets (`BASH_SEARCH_COMMANDS`, `BASH_READ_COMMANDS`, `BASH_LIST_COMMANDS`), and marks the compound command safe only if every non-neutral part is safe. A compound command is pessimistically classified if any part is unknown.

Concurrent Execution: Order Preservation Through Buffering

Given that some tools can run concurrently, the question is: how do you preserve conversation coherence when results arrive out of order?

The answer is a buffering strategy. Tools are partitioned into concurrent batches using a greedy, order-preserving algorithm. Given `[Read, Read, Grep, Edit, Read]`, the first batch is `[Read, Read, Grep]` (all safe), the next is `[Edit]` (exclusive write), the next is `[Read]` (safe again). Consecutive safe tools merge; any write tool breaks the run and gets exclusive access.

Within a concurrent batch, tools execute in parallel. Results are buffered as they arrive. Results are *yielded* to the conversation in original request order, not completion order. The model always sees `[Read result 1, Read result 2, Grep result]` in that order, even if Grep finished first.

This preserves the mental model the model uses when reasoning about its own actions. If it asked for three things, it gets them back in the order it asked.

Fork Agents: Cache Sharing as Architecture

When Claude Code needs to run multiple agents in parallel on similar tasks, it uses a fork pattern. The parent agent spawns children that receive:

- The parent's exact rendered system prompt (not recomputed)
- The parent's exact tool array (byte-identical serialization)
- The full conversation history with a constant placeholder for any unresolved tool results
- A per-child directive appended at the cache boundary

The placeholder strategy is elegant. Instead of different tool results for each child (which would produce different cache keys), all children receive the string `"Fork started -- processing in background"` as the result for any pending tools. This is semantically meaningless but cache-optimal: it unifies the prefix across all children.

The economics work out like this. Assume 5 parallel forks with a 48,500-token shared prefix and 200 tokens of per-child directive. Child 1 pays full price (cache cold). Children 2–5 each pay roughly 10% on the 48,500 shared tokens plus full price on 200 unique tokens. At scale, with warm cache, children 2–5 cost roughly the same as a 5,200-token request instead of a 48,700-token request. The savings scale with context length and child count.

Note

Fork agents include a `<fork-boilerplate>` tag in their system prompt that explicitly overrides any instruction to default to forking. This prevents recursive fork explosions. A regex scan of the message history serves as a secondary guard in case the tag does not survive context compaction.

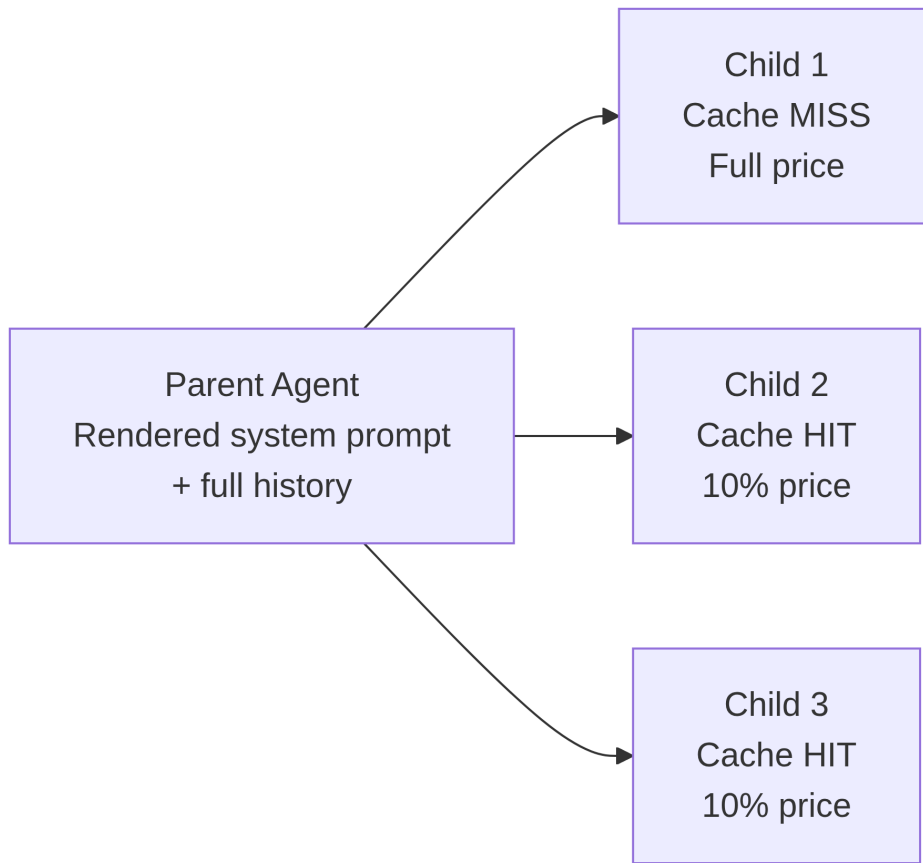


Figure 2: Fork agent cache sharing: children 2-5 pay ~10% of prefix cost after child 1 warms the cache

Context Management: Four Layers Before Giving Up

Context overflow is not handled by a single fallback. There is a four-layer compression hierarchy, each progressively more aggressive:

1. **Tool result budgeting** – per-message size limits (BashTool: 30K chars, GrepTool: 100K chars). Results exceeding limits are persisted to disk; the model receives a `<persisted-output>` wrapper with a preview. No context is consumed by the full result.
2. **Snip Compact** – oldest conversation messages are summarized and replaced. Implemented as a fork: a child agent summarizes the dropped span, the parent receives the summary. Original messages discarded.
3. **Microcompact** – specific tool results identified by ID are replaced with compressed versions. Surgical, not wholesale.
4. **Context Collapse** – the full conversation is summarized from scratch, producing a fresh context that contains the essential history in far fewer tokens.

Only if all four layers fail to resolve the overflow does the agent surface an error to the user.

The loop also includes **death spiral guards**: a `hasAttemptedReactiveCompact` one-shot flag, a maximum of three recovery attempts per error category, a circuit breaker on three consecutive auto-compact failures, and no stop hooks fired on error responses (to prevent hooks from triggering additional API calls mid-error).

The Terminal UI Nobody Sees

Claude Code renders in a terminal using React components via a library called Ink. This is already unusual: React’s virtual DOM, reconciliation, and component model, running against a terminal cell grid instead of a browser DOM.

What makes the rendering layer remarkable is what happens below Ink.

A 200x120 terminal contains 24,000 cells. Naive object allocation – one JavaScript object per cell, per frame, at 60fps – produces 5.76 million object allocations per second. The garbage collector handles this until it does not, at which point frames stutter.

The solution: **packed integer arrays**. Each cell is two `Int32` values. The first stores a character ID (an intern table maps characters to IDs; ASCII characters are fast-pathed through a 128-entry array). The second packs style ID, hyperlink ID, and character width into bit fields.

Comparing two frames then becomes two integer comparisons per cell instead of string equality and object deep-comparison. The diff that drives output to the terminal is a tight loop over integers.

Additional optimizations:

- **Double buffering with pointer swap** – after rendering, `backFrame = frontFrame; frontFrame = newFrame`. No allocation, no GC pressure.
- **Damage rectangle tracking** – only rows touched by the current render are examined during diff. A 200x120 terminal with a 5-row update skips 115 rows entirely.
- **OffscreenFreeze** – React subtrees for scrolled-out messages are frozen. Their timers (spinners, elapsed counters) stop firing. Prevents off-screen content from triggering full-screen resets.
- **Sync update markers** – atomic `ESC[?2026h` (begin buffering) / `ESC[?2026l` (flush) sequences eliminate visible tearing on terminals that support them.

Pool strings are reset every 5 minutes to prevent unbounded growth, with a migration pass that re-interns the live cells from the current front frame into fresh pools.

Memory Without a Database

Claude Code has a persistent memory system. It stores what it learns about users, projects, and preferences across sessions. The surprising part is how it retrieves relevant memories: not with vector embeddings, not with a database index, but with a language model side-query.

The retrieval pipeline works like this. On each turn, a parallel task scans memory file headers (the first 30 lines of each file), assembles a manifest, filters out memories already surfaced in this session, and sends the manifest to a model (Claude Sonnet) with the question: “Given this conversation, which of these memories are relevant?”

The model returns a list of file paths. Those files are read in full and injected into context.

Why use an LLM for recall instead of embeddings? Several reasons:

- The model understands context, not just similarity. It can reason that a memory about “prefers Go over Python” is relevant to a Python code question.
- It handles negation naturally. “Do not add type annotations” is semantically distant from “type annotations” but the model correctly identifies the relevance.
- It deduplicates. The model notices when a memory says something already visible in the conversation and skips it.

The memory taxonomy enforces a discipline that prevents bloat. Memories are classified into four types: *user* (role, expertise, preferences), *feedback* (guidance and corrections), *project* (ongoing work context), and *reference* (pointers to external systems). Code patterns, file paths, and git history are explicitly excluded because they are derivable from current project state. Saving them would fill memory with stale facts that the agent can always recover by reading the files directly.

Tip

Memories include a modification time that is formatted as human-readable age (“47 days ago” rather than an ISO timestamp). This wording triggers better staleness reasoning in the model. A memory tagged “47 days ago” about a function that may have been renamed gets treated with appropriate skepticism. An ISO timestamp does not trigger the same reasoning.

The Five Architectural Bets

Looking across all 18 chapters, five non-obvious bets define the architecture. Each one sacrifices something conventional to gain something specific.

Bet 1: Generator loop over callbacks. A single 1,730-line `while(true)` generator handles every agent interaction. This is a single point of failure but a single point of understanding. Every path through the system is visible in one function. Terminal states are encoded in the return type.

Bet 2: Files over databases. The memory system, the task registry, agent transcripts, hook configurations – all stored in plain files on disk. No SQLite, no Redis, no vector store. Files are human-readable, version-controllable, inspectable with standard tools, and durable across crashes. The query cost (LLM side-call for recall) is accepted as worthwhile.

Bet 3: Self-describing tools. Every tool carries its own schema, safety metadata, permission logic, and rendering logic. Adding a new tool requires no central registry update. MCP tools wrap to the same interface and become indistinguishable from built-ins. The cost is a large `ToolUseContext` object (40+ fields) that every tool receives.

Bet 4: Fork agents for cache sharing. The complexity of threading rendered system prompts, placeholder tool results, and boilerplate recursion guards is accepted in exchange for the 90% token discount on shared prefixes. The economics make this non-negotiable at scale.

Bet 5: Process-isolated hooks over plugins. Third-party extensions run as subprocesses, not as in-process plugins. A crashing plugin cannot take down the agent. It crashes its own process, returns a non-zero exit code, and the agent continues. The cost is IPC overhead and the constraint that hooks must communicate through stdin/stdout and exit codes.

What Makes This Architecture Worth Studying

Claude Code was not designed from a whiteboard. It was evolved under production constraints, with hundreds of thousands of users, where a wrong caching decision costs real money and a bad terminal diff costs real frames.

Several patterns are immediately transferable:

Encode decisions in the type system. Discriminated union return types from generators make failure modes explicit and exhaustive. The compiler enforces that every terminal state is handled.

Fail-closed defaults. New tools default to serial, non-concurrent, read-write. Forgetting to classify a tool conservatively defaults to safe behavior. Hooks require exit code 2 for enforcement; accidental non-zero exits do not accidentally block operations.

Measure the constraint before optimizing. Every optimization in this codebase – sticky latches, fork caching, speculative execution, packed integers

– was justified by a measurable cost. The startup profiler has 50+ checkpoints. Token budgets are tracked per API call. Cache hit rates are observable.

Asymmetric channels for asymmetric access patterns. Reads and writes use different transport mechanisms. Infrastructure state and UI state live in different stores with different initialization constraints. Bootstrap loads before React. Each separation was made because coupling those concerns created specific, observable bugs.

The generator pattern for lifecycle management. Async generators as agent loops, sub-agent spawning, tool execution batches, and rendering schedules. `yield` provides natural backpressure. Cancellation is `iterator.return()`. Terminal reasons are typed. This pattern appears in at least six places in the architecture because it solves the same problem each time: clean lifecycle management with exhaustive terminal state encoding.

Closing Thought

The most instructive thing about this architecture is not any single trick. It is the evidence that sustained constraint – every millisecond costs something, every token costs something – eventually produces elegant solutions through selection pressure rather than upfront design.

The sticky latch is not a clever idea someone had. It is the obvious solution once you have measured that mid-session feature flag toggling costs 50,000 tokens. The packed integer terminal buffer is not premature optimization. It is the response to a profiler showing 5.76 million allocations per second causing GC stalls.

Good architecture, at this scale, is mostly good measurement followed by the simplest solution that fits the constraint.